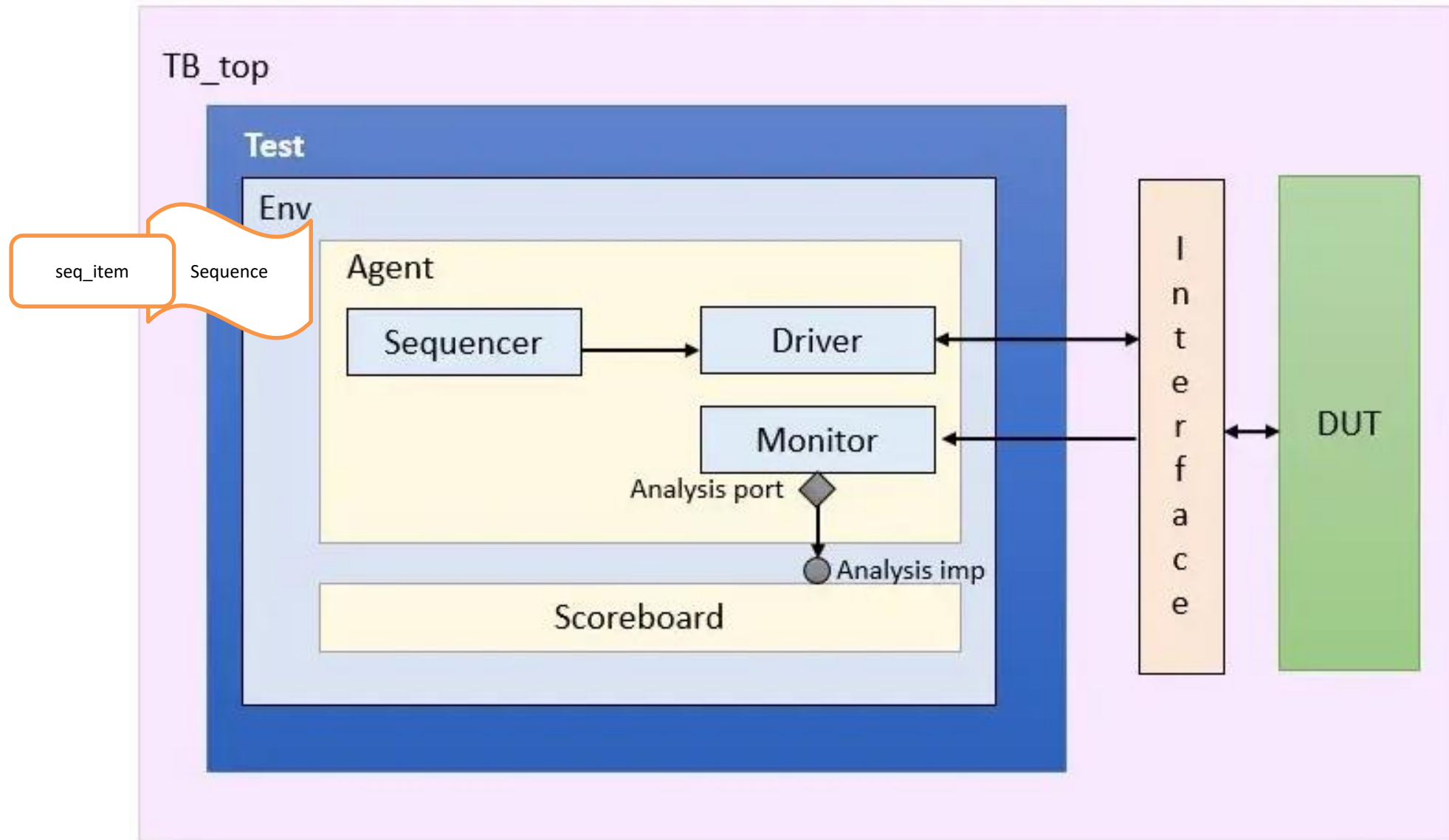


UVM Sequence and Sequencer + Virtual Interface in UVM

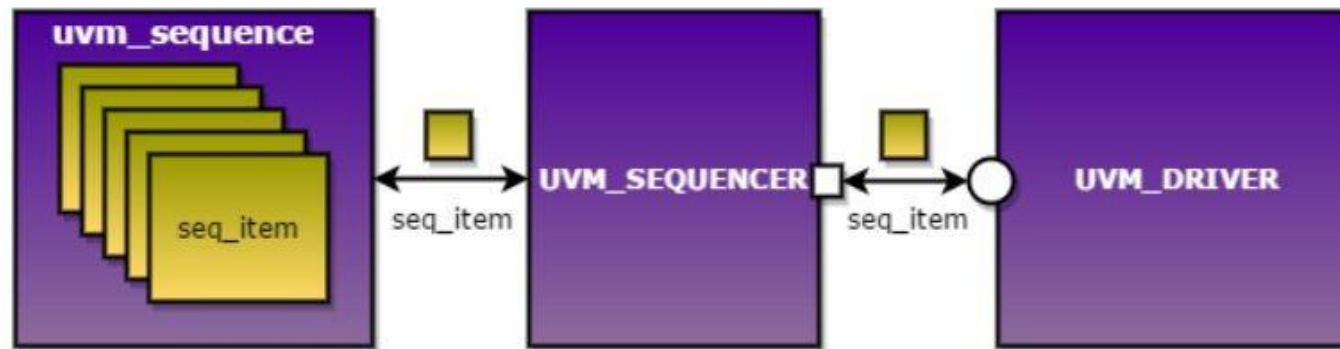
Yasir Farooq

Recall the UVM based Testbench



UVM Sequences

- UVM provides **uvm_sequence** base class, so the user defined sequence class should be extended from this base sequence class
- This class is used to generate the sequences/stimulus and sends the sequences to driver via sequencer
- Sequences can be generated using
 - UVM sequence methods
 - UVM sequence macros
- This class is parametrized with the sequence item class

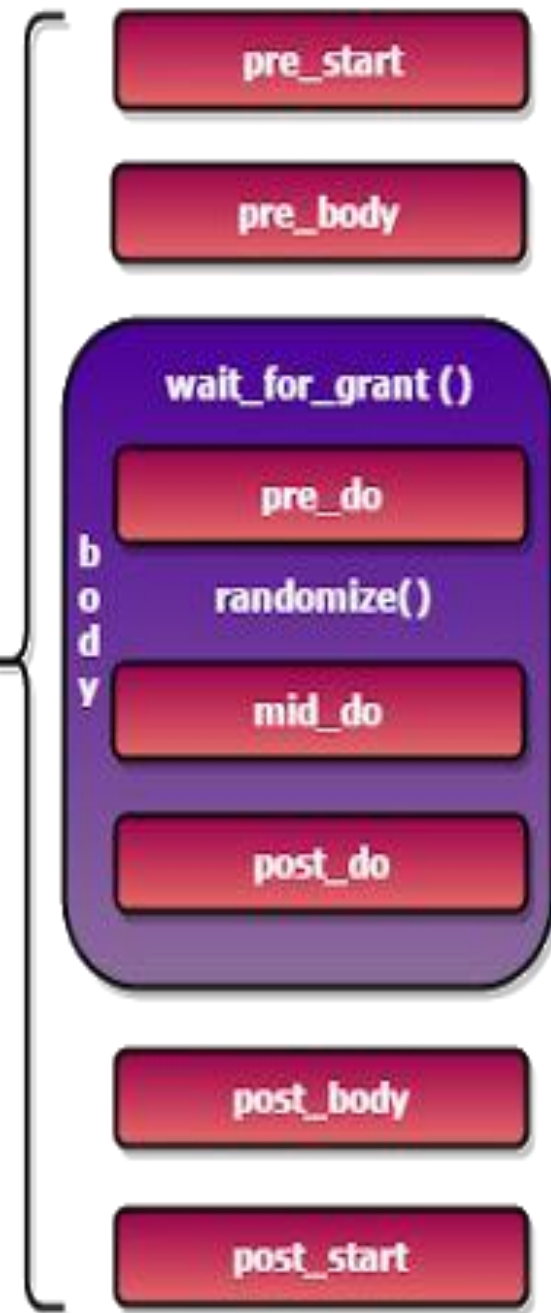


UVM Sequence

Starting a Sequence

- The sequence will get executed upon calling the start of the sequence from the test
 - `sequence_name.start(sequencer_name);`
- Diagram is showing the order in which the methods will get called on calling the start of a sequence
- **mid_do** and **post_do** are functions, all other are tasks
- Logic to generate and send the sequence_item will be written inside the **body()** method of the sequence

`sequence.start(sequencer);`



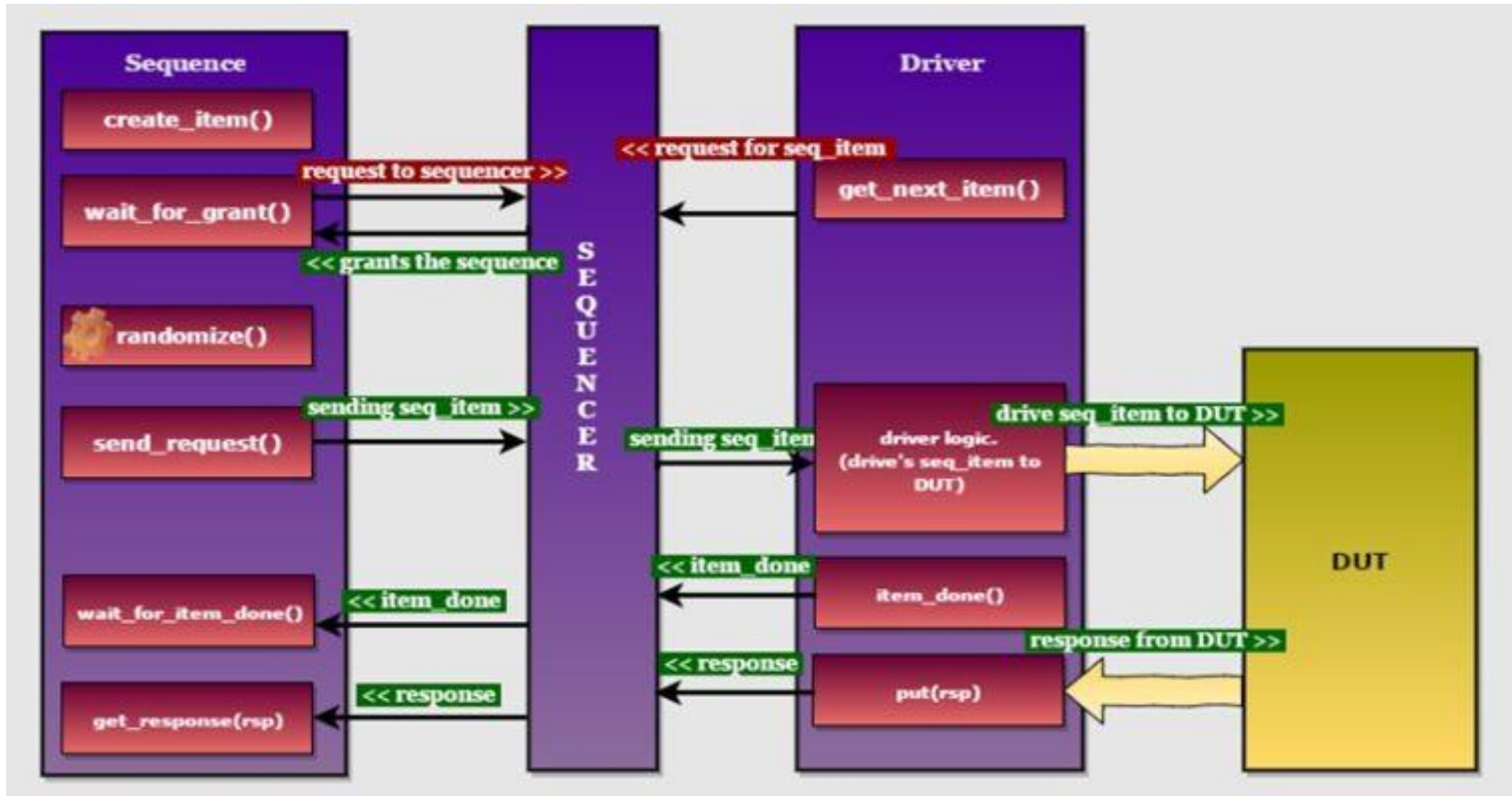
UVM Sequence Methods

- Communication between sequence and driver involves the following steps/methods:
 1. creating seq_item
 2. wait_for_grant()
 3. randomize()
 4. send_req()
 5. wait_for_item_done()
 6. get_response()
- We can use start_item(<item>) and finish_item(<item>) methods as well

*5 and 6 are optional

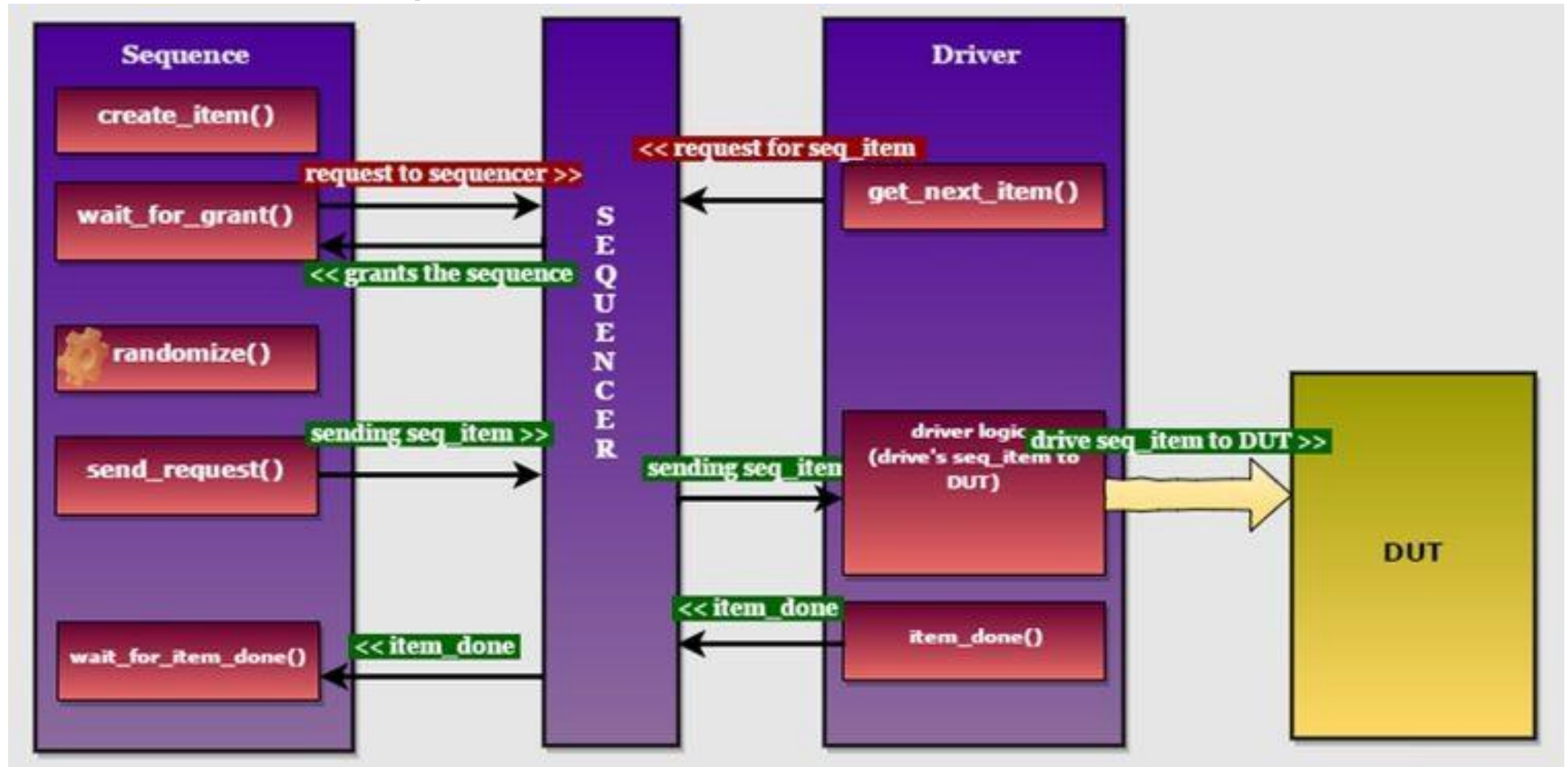
Sequence-Sequencer-Driver Handshake

- With DUT response



Sequence-Sequencer-Driver Handshake

- Without DUT response



Sequence – Example Code

```
class alu_sequence extends uvm_sequence#(alu_transaction);
  `uvm_object_utils(alu_sequence)
  // default constructor
  function new (string name = "alu_sequence");
    super.new(name);
  endfunction
  alu_transaction alu_sq;
  // task body function
  virtual task body();
    repeat (15) begin
      alu_sq = alu_transaction::type_id::create("alu_sq");
      start_item(alu_sq);
      if (!alu_sq.randomize())
        `uvm_fatal("NO_VIF",{"Randomization Not Happen: ", get_full_name(),".aluIF"})
      `uvm_info("ALU_Sequence",alu_sq.sprint(),UVM_LOW)
      finish_item(alu_sq);
    end
  endtask
endclass
```

UVM Sequence Macros

- UVM also provides sequence macros alternate to sequence methods
 - This makes the code small in size
- These macros are used to start sequences and sequence items on default sequencer, m_sequencer
 - Sequence Action Macros
 - ``uvm_create(item/seq)`
 - ``uvm_do(item/seq)`
 - ``uvm_do_pri(item/seq, priority)`
 - ``uvm_do_with(item/seq, constraints)`
 - ``uvm_do_pri_with(item/seq, priority, constraints)`

UVM Sequence Macros

- Sequence on Sequencer Action Macros
 - ``uvm_create_on(item/seq, seqr)`
 - ``uvm_do_on (item/seq, seqr)`
 - ``uvm_do_on_pri (item/seq, seqr, priority)`
 - ``uvm_do_on_with(item/seq, seqr, constraints)`
 - ``uvm_do_on_pri_with(item/seq, seqr, priority, constraints)`
- Sequence Action Macros for Pre-Existing Sequences
 - ``uvm_send(item/seq)`
 - ``uvm_send_pri(item/seq, priority)`
 - ``uvm_rand_send(item/seq)`
 - ``uvm_rand_send_pri(item/seq, priority)`
 - ``uvm_rand_send_with(item/seq, constraints)`
 - ``uvm_rand_send_pri_with(item/seq, priority, constraints)`

Sequence – UVM Sequence Macros

These macros are used to start sequences and sequence items on default sequencer, `m_sequencer`. Some these macros are explained below:

- ``uvm_do()`: Takes a `seq_item` or sequence as an argument and executes all 6 steps described above
- ``uvm_create()`: Creates an item or sequence
- ``uvm_send()`: Executes all steps except `create()` and `randomize()`

Sequence – UVM Sequence Macros (cont'd)

- **`uvm_rand_send():** Only create() is skipped, rest are executed in order
- **`uvm_do_with():** Performs all six steps and also takes constraints as second argument
- **`uvm_rand_send_with():** create() is skipped, all other methods are executed in order and takes constraints as the second argument

Sequence – Example Code (using macros)

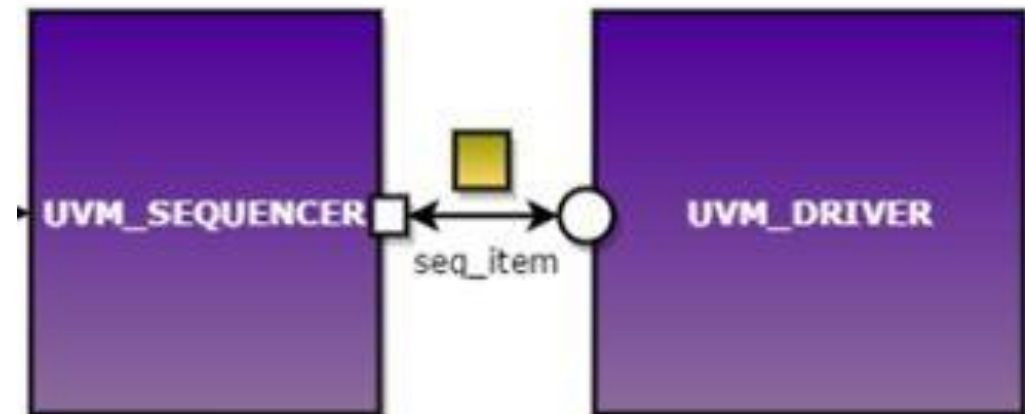
```
class add_sequence extends uvm_sequence#(alu_transaction);
  `uvm_object_utils(add_sequence)

  alu_transaction alu_sq;
  function new (string name = "add_sequence");
    super.new(name);
  endfunction

  virtual task body();
    alu_sq = alu_transaction::type_id::create("alu_sq");
    `uvm_do_with(alu_sq,{alu_sq.op==1; alu_sq.A ==5;})
    `uvm_info("ALU_Sequence",alu_sq.sprint(),UVM_LOW)
  endtask
endclass
```

UVM Sequencer

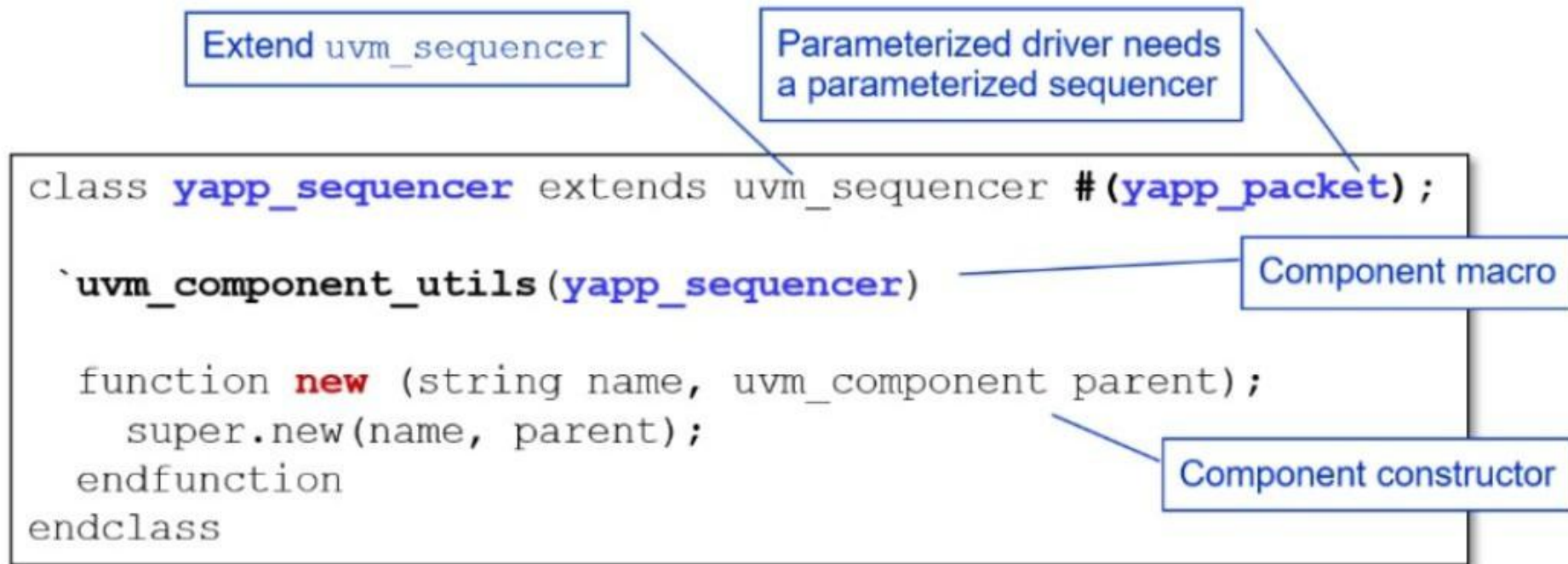
- Always extended from `uvm_sequencer` base class
- It controls the flow request and response sequence items between uvm sequence and uvm driver
- **uvm_sequencer** and **uvm_driver** base classes have **seq_item_export** and **seq_item_port** defined respectively. Users only need to connect them using TLM connect method
 - `driver.seq_item_port.connect(sequencer.seq_item_export);`
 - This is done in agent class



Sequencer – Example Code

```
class alu_sequencer extends uvm_sequencer#(alu_transaction);  
  
  `uvm_component_utils(alu_sequencer)  
  //-----  
  //constructor  
  //-----  
  function new(string name, uvm_component parent);  
    super.new(name,parent);  
  endfunction  
  
endclass
```

Sequencer – Example Code



UVM Sequencer

By default sequencer does not generate a stimulus

- A sequence data item defines transactions for sequencer
- Many sequences can be defined for a given sequencer
- One must execute a sequence on the sequencer to apply its stimulus

m_sequencer and p_sequencer

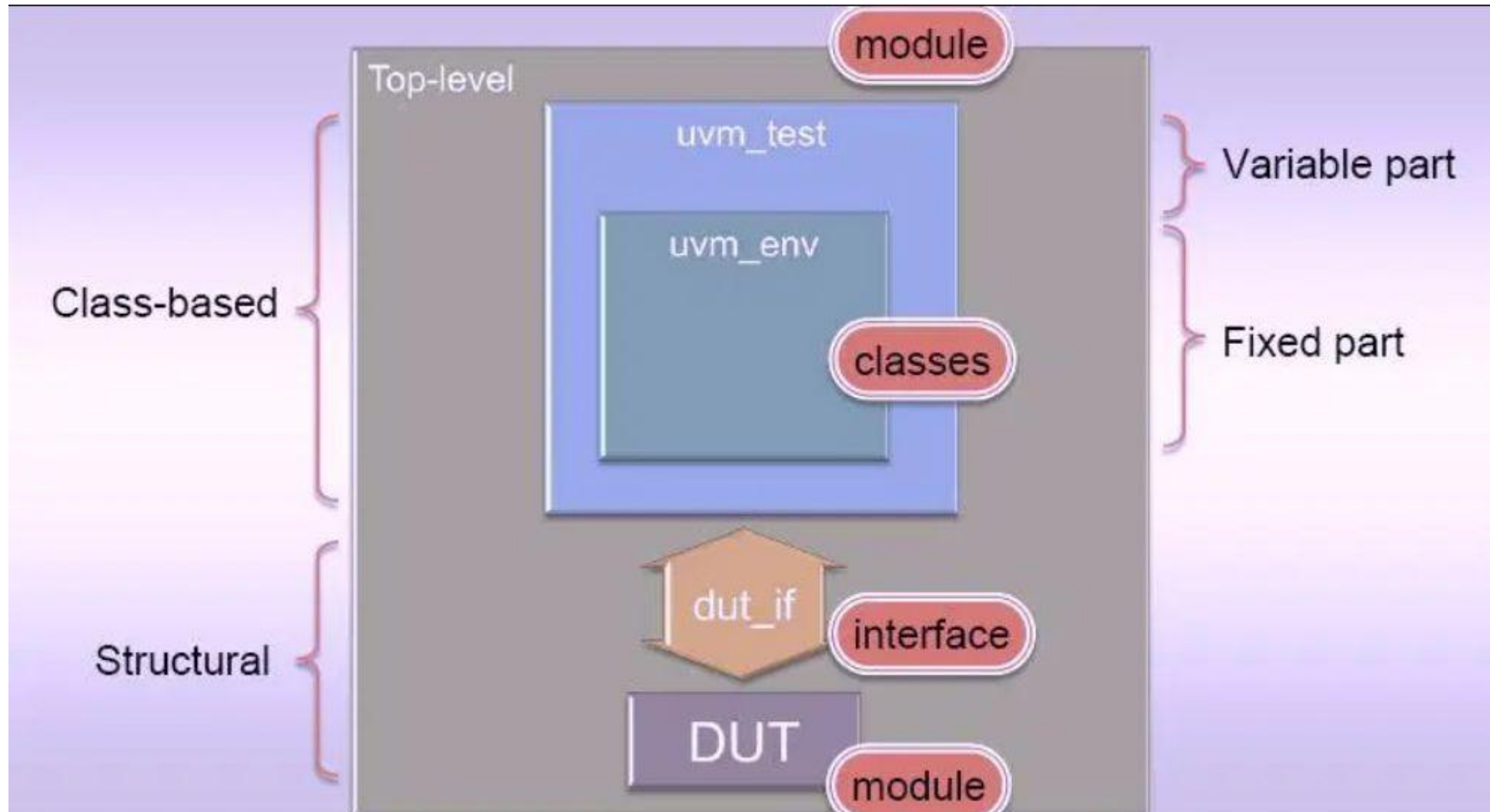
- **m_sequencer:**
 - The m_sequencer handle contains the reference to the sequencer(default sequencer) on which the sequence is running. This is determined by,
 - the sequencer handle provided in the start method
 - the sequencer used by the parent sequence
 - the sequencer that was set using the set_sequencer method
- **p_sequencer:**
 - The p_sequencer is a variable, used as handle to access the sequencer properties
 - p_sequencer is defined using the macro
``uvm_declare_p_sequencer(SEQUENCER_NAME)`

Virtual Interface in UVM

Virtual Interface in UVM

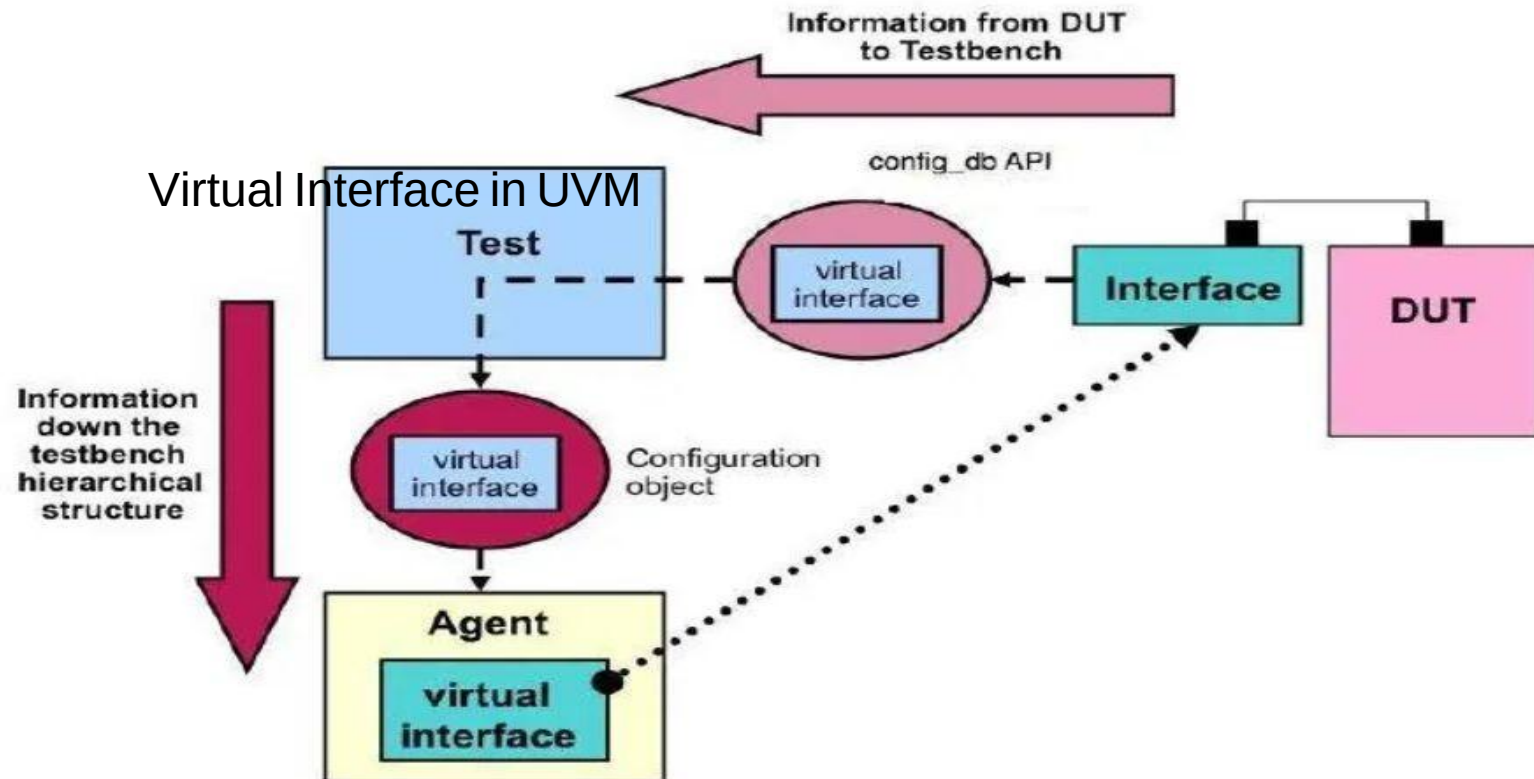
- In traditional testbenches/verification environments all components are static in nature but this is not the case for UVM
 - To maintain the reusability of UVM, not all components are connected with the DUT
- In UVM environment, the test class receives this virtual interface information from the DUT
 - Later the test class propagates this virtual interface (interface instance handle) information to the physical components
 - e.g., Driver, Monitor
- Whole of this process happens by utilizing the “uvm_config_db” API
 - set() and get() methods

Virtual Interface in UVM



Virtual Interface in UVM Example

<https://www.edaplayground.com/x/Xk86>



Lab Exercise

Lab 7 **Using UVM Sequence and Understanding Virtual interface**

To learn the generation of meaningful directed and random sequences.

To learn setting and getting of virtual interface in UVM